

RAPOR

Bu projede, ana veri yapısı bir altıgen ile temsil edilen bir öncelik kuyruğu olan bir sistem geliştirdim; burada altıgenin her bir kenarı (düğümü) bir ikili arama ağacını tutmaktadır. Temel amaç, bir altıgenin bir sonraki altıgene bir ağaç göndermesini ve bir önceki altıgenden bir ağaç almasını sağlamaktır.

Bunun için öncelikle en içteki veri yapısı olan ağacı ("BST") geliştirdim. Bunun içine, öncelikli ağacı elde etmek için "height", bir ağacın kökünü elde etmek için "peek root" ve "post order" gibi gerekli bazı metotları ekledim. "post order" metodunun yürütülmesi sırasında, ağacın tüm düğümlerini saklamak için ayrı bir "Helper.hpp" dosyasında uygulanan "save node of tree" sınıfından "save node" (düğümü kaydet) adlı statik bir fonksiyonu çağırıyorum. "Helper.hpp" dosyasından daha sonra bahsedeceğim.

Ardından, ikinci veri yapısı olan altıgeni (öncelik kuyruğu) oluşturdum ve içinde 6 boyutunda (bir altıgenin 6 kenarı vardır) bir ağaç dizisi ve her zaman 0. konumu işaret etmesi için "front" adını verdiğim bir değişken tanımladım. Bunu yaptım çünkü listeden bir ağaç çıkarıldığında, sağındaki tüm ağaçları sola kaydırıyorum, böylece çıkarılan ağacın bıraktığı boşluk doldurulur ve son pozisyonlar boş kalır.

Altıgen listeden bir ağaç çıkarmak için, parametre olarak ağacın çıkarıldığı turu alan "remove tree" metodunu oluşturdum. Eğer tek sayılı bir tur ise, ağaç (BST) sınıfının "post order" metodunu çağırıyorum ve daha önce belirtildiği gibi, ilk ağacın düğümlerini "helper" sınıfında saklıyorum. Eğer çift sayılı bir tur ise, listedeki en yüksek önceliğe sahip ağaç ile "post order" metodunu çağırıyorum. Ağaç verilerini çıkardıktan ve kaydettikten sonra onu listeden siliyorum.

Altıgen sınıfında ayrıca "receive tree" metodunu da uyguladım. Bu metotta, "save node of tree" sınıfının (çıkarılan ağacın tüm düğümlerini saklayan) "peek and delete" metodunu çağırıyorum ve düğümleri 0. pozisyondan başlayıp son pozisyona kadar ilerleyerek bir sonraki altıgene dağıtıyorum. Eğer bir pozisyon boşsa, önce o pozisyonda ağacı oluşturuyor, sonra düğümü ekliyorum.

Son olarak, tüm altıgenleri birbirine bağlı tutan en dıştaki veri yapısını, yani basit bir dairesel bağlı listeyi oluşturdum, böylece altıgenler arasındaki ağaç alışverişi doğru bir şekilde çalışır.

Projenin yapısı şu şemada özetlenebilir: $BST \subset Priority\ Queue \subset Circular\ Linked\ List$.

Ağaçlar bir metin dosyasından çıkarılmaktadır. Dolayısıyla program başladığında bu dosyayı okuyor, ağaçları oluşturuyor ve onları altıgenlere ekliyorum. Proje sırasında yaşadığım zorluklardan biri altıgenlerin ekrana basılmasıydı, çünkü zig-zag şeklinde basılmaları gerekiyordu ve "helper" sınıfı bu konuda da bana yardımcı oldu.

"Helper" Dosyasının Açıklaması:

Bir ağacı silerken, düğümlerini saklamakta zorluk yaşıyordum. Bu yüzden bir "Helper.hpp" oluşturdum ve içinde silinen ağacın tüm düğümlerini kaydetmek için statik alanlara sahip

temel bir kuyruk oluřturdum. Bir kuyruk uyguladım çünkü düğümleri bir sonraki altıgene dağıtırken giren ilk düğümün çıkan ilk düğüm olması gerekiyordu. Bu kuyruk sınıfında iki ana statik metot vardı:

- **"save node"**: Tüm düğümleri kaydetmek için "BST" sınıfının "post order" metodunda çağrılır.
- **"Peek and delete"**: Altıgen sınıfının "receive nodes" metodunda çağrılır. Altıgene ilk düğümü döndürür ve hemen ardından onu kuyruktan siler. Döngü, kuyruk boşaldığında sona erer ve yeni düğümleri kaydetmek için "BST" sınıfında tekrar kullanılmaya hazır hale gelir.

Ayrıca altıgenleri zig-zag şeklinde (her satırda 6 eleman) yazdırmakta da zorlandım. Bu yüzden "Helper" içinde ikinci bir sınıf oluřturdum: bir stack. Bir "stack" oluřturdum çünkü çift satırlarda (ikinci satır, dördüncü satır...) basılacak altıgenlerin tersten (sondan başa) basılması gerekiyordu. "stack" tam olarak bunu yapar: altıgenleri doğru sırada yazdırmak yerine, onları stack'te saklıyorum ve oradan ekrana ters sırada yazdırıyorum, çünkü giren son eleman basılan ilk elemandır. Her yazdırma işleminde altıgen stack'ten silinir. Böylece, o çift satırdaki tüm altıgenler yazdırıldıktan sonra, stack boşalır ve bir sonraki çift satırda tekrar kullanılmaya hazır hale gelir.

Bu proje ile farklı veri yapılarıyla tek bir sistemde çalışma becerilerimi geliřtirebildim ve bir altıgeni silmenin hafızada sakladığı ağacı silmek anlamına gelmediğini fark ederek işaretçilerin nasıl çalıştığını daha iyi anladım. Bu nedenle, silme işlemi (destructors) en iç yapıdan en dış yapıya doğru gerçekleşir.